Carleton University

Department of Systems and Computer Engineering

SYSC 4907 - Capstone Project

Smart Scheduler for Carleton Engineering Students

Final Report

Group G67

Anas Ayoubi (101226450)

Hakeem Khan (101233039)

Pietro Adamvoski (101238885)

Safiullah Rattar (101245709)

Supervisor: Prof. Jerome Talim

A report submitted in partial fulfillment of the requirements

of SYSC 4907 Engineering Project

Faculty of Engineering

Carleton University

Abstract

The Smart Scheduler is a web application that helps Carleton students plan their course timetables. It parses a student's academic audit, checks prerequisite eligibility, and generates conflict-free schedules from the available course sections. We built it as a full-stack system with a React and TypeScript frontend, an Express and Node.js backend, and a PostgreSQL database holding the course, section, and prerequisite data. This report explains the requirements, design decisions, implementation, and testing work behind the project. It also covers the features we cut, the parts that still feel unfinished, and what we learned over the two terms.

Contents

1 Introduction 1

1.1 Motivation 1

1.2 Project Objective 2

1.3 Report Overview 2

2 The Engineering Project 4

2.1 Health and Safety 4

2.2 Engineering Professionalism 4

2.3 Project Management 5

2.4 Justification of Suitability for Degree Program 6

2.5 Individual Contributions 7

2.5.1 Project Contributions 7

2.5.2 Report Contributions 8

3 Requirements and Design 9

3.1 Original Requirements 9

3.2 System Architecture 12

3.2.1 Architectural Decision: Separated Frontend and Backend 12

3.2.2 High-Level Architecture 13

3.2.3 Design Patterns 14

3.3 Database Design 15

3.3.1 Schema Design Process 15

3.3.2 Final Schema 16

3.3.3 Data Population 17

3.4 Descoped Features and Rationale 17

4 Implementation 19

4.1 Prerequisite Validation Engine 19

4.1.1 Problem Statement 19

4.1.2 Data Structure 19

4.1.3 Algorithm 20

4.1.4 Grade Comparison 21

4.1.5 Design Decision: Recursive vs. Iterative 21

4.2 Schedule Generation Algorithm 22

4.2.1 Problem Definition 22

4.2.2 Algorithm Selection 22

4.2.3 Implementation 23

4.2.4 Conflict Detection 24

4.2.5 Performance 24

4.3 Academic Audit Parsing 25

4.3.1 HTML Parsing 25

4.3.2 PDF Parsing 26

4.3.3 Design Decision: `pdftotext` vs. JavaScript Library 26

4.4 Backend API 27

4.5 Frontend Interface 29

4.5.1 Page 1: Academic History Import 29
4.5.2 Page 2: Course Selection 29
4.5.3 Page 3: Schedule Review 30
4.5.4 UI Technology 30
5 Testing and Evaluation 31
5.1 Testing Approach 31
5.1.1 Manual Functional Testing 31
5.1.2 CLI Testing Tool 32
5.2 Requirements Verification 32
5.3 Limitations of Testing Approach 33
6 Reflections 35
6.1 Achievement of Original Objectives 35
6.2 What Changed and Why 36
6.3 Team Dynamics 37
6.4 Hindsight Decisions 37
6.5 Overall Assessment 38
7 References 40
H Appendix A: Original Project Proposal 42
I Appendix B: API Reference 43
I.0.1 Schedule Generation Request Example 43
I.0.2 Schedule Generation Response Example 43
J Appendix C: Database Statistics 45

List of Tables

Table 1 Functional Requirements and Final Status 10

Table 2 Non-Functional Requirements and Final Status 11

Table 3 Database Schema Summary 16

Table 4 Schedule Generation Performance 24

Table 5 REST API Endpoints 28

Table 6 Functional Requirements Verification Matrix 33

List of Appendices

- Appendix A: Original Project Proposal
- Appendix B: API Reference
- Appendix C: Database Statistics

1 Introduction

1.1 Motivation

Every term, Carleton students go through the same routine: open your academic audit, open the undergraduate calendar, open the timetable, and try to piece together a schedule that actually works. Carleton Central is an enrollment system, not a planning tool. It does not tell you which electives you are still eligible for, and it does not help you work through the larger planning problem of building a timetable that fits your degree progress and your actual life.

Part of the problem is that prerequisite rules at Carleton are not always simple. Some courses require specific combinations of other courses, minimum grades, or year standing. Checking these by hand means flipping between multiple pages and hoping you did not miss a condition buried three levels deep in an AND/OR expression. It gets even harder for students who are not following the traditional path through their program tree, whether that is because of co-op sequencing, repeated courses, transfer credits, late program changes, or simply taking courses in a different order than the default progression.

Existing tools do not really solve this. Carleton's public Course Scheduler lets you visualize timetables, but it knows nothing about your academic history. Third-party tools like Coursicle have the same limitation. The result is that students end up consulting three or four different sources, doing the cross-referencing in their heads or on a spreadsheet, and spending a lot of time brute-forcing combinations that fit their required courses, elective options, work hours,

and other constraints. Even after all that effort, there is still the question of whether they actually found every valid combination or whether they missed a schedule that would have fit better.

1.2 Project Objective

We set out to build a web application that connects a student's academic record to the course registration process. In concrete terms, the system needed to:

1. Parse a student's academic audit (HTML or PDF) to extract completed courses and figure out what they still need.
2. Check prerequisite eligibility for any course, using a structured representation of Carleton's prerequisite rules.
3. Generate all conflict-free timetable combinations for a set of courses, handling lectures, labs, and tutorials.
4. Show the results in an interactive timetable that students can browse and compare.

The application was scoped to run locally. We were not trying to integrate with Carleton Central's live registration system. The goal was to give students the Course Registration Numbers (CRNs) they need to enroll manually.

1.3 Report Overview

The rest of this report is organized as follows. Chapter 2 covers the standard engineering project considerations required by the department: health and safety, professionalism, project management, suitability for the Software Engineering degree, and who did what. Chapter 3 goes through the requirements and design, including what we originally proposed, what alternatives we considered, and why

we made the architectural choices we did. Chapter 4 gets into the implementation details for the prerequisite validator, scheduling algorithm, audit parser, database, API, and frontend. Chapter 5 covers testing and evaluation. Chapter 6 is a reflection on the project: what went well, what did not, and what we would change. The original project proposal is in Appendix A.

2 The Engineering Project

2.1 Health and Safety

This is a software project. There is no hardware, no lab equipment, and no physical safety risk. The main concern is data privacy: students upload their academic audits, which contain their name, student ID, and course history. We handle this by processing uploaded files in memory and not writing them to disk on the server. The parsed data (name, ID, courses, grades) goes into a local PostgreSQL database that only runs on the developer's machine and is not exposed to the internet.

We did not build an authentication system because the application runs locally, not as a public service. If it were ever deployed publicly, authentication and proper access control would be necessary. Similarly, all communication between the frontend and backend happens over plain HTTP on localhost. A production deployment would need HTTPS, which was one of our original non-functional requirements (NFR6) that we did not end up meeting. Section 6 discusses this.

2.2 Engineering Professionalism

We tried to be honest throughout the project, both with our supervisor and with ourselves. When we fell behind on features, we told Prof. Talim in our meetings rather than pretending things were on track. When preference-based ranking turned out to be more work than we could do well in the remaining time, we chose to cut it rather than ship something half-baked. That felt like the right call,

even though it meant the final product does not match the full scope of the proposal.

On the code side, we followed consistent TypeScript conventions, used descriptive names, and added comments where the logic was not obvious. The Prisma schema is the single source of truth for the data model, and the API follows RESTful conventions.

2.3 Project Management

The project ran across two terms (Fall 2025 and Winter 2026). We met weekly to go over what each person had done, talk through anything that was stuck, and figure out the next week's priorities. Between meetings, we communicated through a group chat. We used Git and GitHub for version control, with each person working on their own branch and merging into main.

During the Fall term, we focused on the foundation: backend infrastructure, database schema, the scheduling engine, prerequisite validation, and the initial frontend pages. One of the first real decisions we made was to split the project into a separate frontend (React + Vite) and backend (Express), rather than the unified Next.js app from the proposal. That was a project management decision as much as a technical one. With four people touching the same codebase, we would have spent half our time resolving merge conflicts. The split let us work in parallel without stepping on each other.

In the Winter term, we shifted to integration, PDF audit parsing, elective recommendations, frontend polish, and this report. We tracked remaining work through shared lists and meeting notes instead of a formal issue tracker. For four

people, that mostly worked. But there were times when it was not clear who owned a particular task, and a few things slipped through the cracks because of it. Chapter 6 says more about this.

We did not set up CI/CD. Builds and tests were run manually. For a project of this duration and team size, that was workable, but it would not scale.

2.4 Justification of Suitability for Degree Program

All four team members are in the Software Engineering program, and this project used skills from across the curriculum. Here is how specific courses connected to the work we did:

Software Design and Architecture (SYSC 4120): The separated frontend/ backend architecture, the RESTful API contract, and the ORM-based data access layer are all applications of separation of concerns and layered design. Modeling prerequisites as a tree (using the Composite pattern) rather than flat text required the kind of data modeling we learned in this course.

Algorithms and Data Structures (SYSC 2100): The schedule generator is a constraint-satisfaction problem solved with backtracking search. The prerequisite validator uses recursive tree traversal. We analyzed both for time and space complexity and optimized the scheduler with pruning to get it under two seconds for typical inputs.

Database Management (COMP 3005): We designed the PostgreSQL schema in Third Normal Form with six interrelated models, foreign key constraints, unique constraints, and indexes. Prisma ORM gave us type-safe database access from TypeScript.

Web Application Development (SYSC 4806): The frontend uses React 18 with component-based architecture, state management, and async API calls. The backend uses Express.js with RESTful routing, error handling, input validation, and middleware for file uploads and CORS.

Software Engineering Process: In practice, our process was closest to incremental development with regular feedback. We defined the main requirements early, divided the system into backend, frontend, database, and algorithm components, and then implemented and integrated those pieces over multiple iterations. As the project progressed, we revised priorities, descoped features, and adapted the design, which is consistent with the engineering process methods taught in the class.

2.5 Individual Contributions

2.5.1 Project Contributions

Anas Ayoubi focused on the scheduling algorithm. He implemented the backtracking schedule generator, including support for multi-section courses and time-conflict checking.

Hakeem Khan handled data acquisition. He worked on scraping the source data and converting it into JSON so it could be loaded and used by the rest of the system.

Pietro Adamvoski handled the database schema and prerequisite logic. He worked on the Prisma models, including the database structure used for prerequisites, and contributed to the logic for checking whether a student meets course requirements.

Safiullah Rattar handled PDF parsing and search optimization. He worked on extracting course information from audit PDFs and on improving the search process so schedule generation could run more efficiently.

2.5.2 Report Contributions

- **Chapter 1 (Introduction)**: Pietro Adamvoski
- **Chapter 2 (The Engineering Project)**: All members (collaborative)
- **Chapter 3 (Requirements and Design)**: Pietro Adamvoski, Anas Ayoubi
- **Chapter 4 (Implementation)**: Pietro Adamvoski (database, prerequisites), Anas Ayoubi (scheduling algorithm), Safiullah Rattar (PDF parsing, search optimization), Hakeem Khan (data acquisition)
- **Chapter 5 (Testing and Evaluation)**: All members
- **Chapter 6 (Reflections)**: All members (collaborative)

3 Requirements and Design

This chapter covers what we set out to build and how we decided to build it. The focus is on the design process: what options we considered, why we picked what we picked, and where things changed from the original proposal.

3.1 Original Requirements

The proposal (October 2025) listed nine functional requirements (FR1–FR9) and six non-functional requirements (NFR1–NFR6). Tables 3.1 and 3.2 summarize the requirement categories discussed in the final system and how they ended up.

ID	Requirement	Status	Notes
FR1	Academic Audit Parsing (HTML/PDF)	Implemented	HTML fully supported; PDF supported via `pdftotext` CLI tool
FR2	Prerequisite Validation	Implemented	Tree-based recursive evaluation with grade checking
FR3	Manual Course Entry	Implemented	Search and select interface on Course Selection page
FR4	Elective Recommendation	Partially Implemented	Course suggestion endpoint exists; elective category display on frontend
FR5	Schedule Generation	Implemented	Backtracking CSP solver with multi-section support
FR6	Preference-based Ranking	Not Implemented	Descoped; all valid schedules returned unranked
FR7	Interactive Timetable View	Implemented	Visual weekly grid with color-coded courses
FR8	Error and Conflict Feedback	Implemented	Prerequisite warnings, missing-section alerts, conflict detection

Table 1: Functional Requirements and Final Status

ID	Requirement	Status	Notes
NFR1	Performance (<10s generation, <5s load)	Met	Generation completes in <2s for 5 courses
NFR2	Usability (intuitive first-time use)	Partially Met	Three-step wizard flow; no formal SUS evaluation conducted
NFR3	Reliability (no crashes, graceful error handling)	Met	Try-catch on all routes; fallback to manual entry if parse fails
NFR4	Compatibility (Chrome, Firefox, Safari)	Partially Met	Tested primarily on Chrome; no formal cross-browser testing
NFR5	Maintainability (modular, documented)	Met	Separated modules, Prisma schema, architecture documentation
NFR6	Security (HTTPS)	Not Met	Application runs on localhost over HTTP; no deployment to production

Table 2: Non-Functional Requirements and Final Status

Most of the functional requirements were fully or partially implemented. The main feature we deliberately cut during the Winter term was FR6, preference-based ranking. Section 3.4 explains why.

3.2 System Architecture

3.2.1 Architectural Decision: Separated Frontend and Backend

The proposal called for a single Next.js 14 application. In that setup, the user interface and the server-side logic would both live in the same project instead of being split into separate frontend and backend codebases. We tried that in the first couple of weeks and quickly realized it was going to be painful for a four-person team. Everyone would be editing the same project, bumping into merge conflicts on shared files, and generally getting in each other's way.

So we split it. The frontend became a standalone React 18 app built with Vite and TypeScript. The backend became an Express.js server with its own TypeScript codebase, talking to PostgreSQL through Prisma. The two communicate over a REST API on localhost:3001.

This worked well. The frontend developer could build pages against mock data or a running backend without needing to touch any server code. The backend developers could test endpoints with curl or the CLI tool without opening a browser. Merge conflicts dropped significantly.

Vite's hot-module replacement was also faster in practice. Hot-module replacement is the feature that updates the browser immediately after a code change without rebuilding and reloading the whole page. With Vite, small frontend edits usually showed up almost right away. With Next.js, the feedback

loop felt slower because the development server had more framework overhead and often needed to do more work before showing the change. That speed difference mattered because frontend development involved a lot of small adjustments to layout, state handling, and component behavior. When each edit appears quickly, you can iterate much faster.

The tradeoff was losing Next.js features like server-side rendering, file-based routing, and integrated API routes. But we did not need SSR (this is a client-side tool, not a content site), and the explicit separation was worth more to us than the convenience of a unified framework.

3.2.2 High-Level Architecture

The system has three layers:

1. **Client layer (React + Vite)**: A single-page app that renders the UI, takes user input (audit uploads, course selections), and displays generated schedules. It talks to the backend via HTTP requests to `localhost:3001/api`.
2. **Server layer (Express + Node.js)**: The application server with the business logic. It has four groups of routes: courses (search, detail, sections), students (profile, prerequisite validation, eligibility), schedules (generation), and audit (upload, parse, course suggestion). It accesses the database through Prisma Client.
3. **Data layer (PostgreSQL + Prisma)**: A relational database with all the course catalog data, section schedules, prerequisite rules, and student records. Prisma handles type-safe queries and schema migrations.

There is also a **CLI layer** for development and testing. Command-line scripts let you test the prerequisite validator, audit parser, and schedule generator without running the web app. This turned out to be one of the more useful things we built, because it let us iterate on algorithm logic much faster than going through the browser every time.

3.2.3 Design Patterns

A few standard patterns show up in the codebase:

MVC: The backend roughly follows Model-View-Controller. Prisma models are the data layer, Express route handlers are the controllers, and JSON responses are the view (there is no server-side HTML).

Repository pattern: All database access goes through Prisma Client. Route handlers never contain raw SQL. This keeps the data access layer testable and would make it easier to swap databases later.

Composite pattern: The prerequisite tree uses this. A PrerequisiteNode can be a leaf (COURSE, YEAR_STANDING, PERMISSION) or a composite (AND, OR) with child nodes. This lets us represent arbitrarily complex prerequisite expressions in a uniform way.

Strategy pattern: The audit parser picks a parsing strategy based on file type. The upload endpoint detects whether the file is HTML or PDF and dispatches to the right parser function. Both return the same `StudentAuditData` interface.

3.3 Database Design

3.3.1 Schema Design Process

The database schema went through several iterations. The first version stored prerequisites as plain text strings (e.g., "SYSC 3110 and SYSC 3020 or COMP 3005"). Easy to implement, impossible to evaluate programmatically. The system would have needed to parse that natural-language string every time it checked a prerequisite, and the phrasings in the Carleton calendar are inconsistent enough that reliable parsing was not realistic.

We looked at three options:

1. **Flat text**: Store the raw prerequisite string. Simple, but requires runtime parsing of unstructured text. We rejected this because the parsing would be too fragile.
2. **Adjacency list**: Store prerequisites as rows in a junction table (course_id, required_course_id). This handles simple AND/OR cases but cannot represent nested expressions like "(A AND B) OR (C AND D)." Many Carleton courses have prerequisites like this, so we ruled it out.
3. **Tree structure (what we went with)**: Store prerequisites as a tree of typed nodes in a self-referential table. Each node has a type (AND, OR, COURSE, YEAR_STANDING, PERMISSION), optional references to a required course or year level, and a parent pointer. This handles arbitrary nesting and covers every prerequisite pattern we found in the calendar.

The tree structure was the only option that correctly represents the full range of Carleton's prerequisite rules while still being straightforward to evaluate with a recursive traversal.

3.3.2 Final Schema

The database has six models:

Model	Primary Key	Description
`Course`	`id` (e.g., "SYSC4907")	Course metadata: title, credits, description
`Prerequisite`	`courseId` (FK)	Raw prerequisite expression string for display
`PrerequisiteNode`	`id` (generated unique string ID)	Tree node: type, required course/year, parent pointer
`Section`	`id` (generated unique string ID)	Section schedule: CRN, days, times, instructor, location, term
`Student`	`id` (generated unique string ID)	Student profile: student ID, name, program
`CourseCompletion`	`id` (generated unique string ID)	Junction table: student-course with grade

Table 3: Database Schema Summary

The schema is in Third Normal Form. We used the course code (for example, "SYSC4907") as the primary key for `Course` because that is already the identifier used throughout the Carleton calendar and the rest of the application. The other

models use generated IDs because they represent records such as sections, students, prerequisite nodes, and course completions, where there is no single natural key that is as clear or stable. The `PrerequisiteNode` table also uses a self-referential foreign key (`parentId` referencing `id`) so prerequisite expressions can be stored as trees.

3.3.3 Data Population

The database was populated from two sources:

1. **Timetable data**: Course section schedules (CRNs, times, instructors, locations) were provided by the project supervisor in a structured format. We wrote an import script to ingest this data. The database has 1,836 courses with section data and 7,680 individual sections.
2. **Prerequisite data**: We scraped prerequisite rules from the Carleton undergraduate calendar with a Node.js script. The scraper walked the calendar site, pulled out the prerequisite text for each course, and parsed it into tree form. The database has 322 courses with parsed prerequisite trees, 348 courses total.

Appendix C gives a compact summary of the final database statistics.

3.4 Descoped Features and Rationale

FR6: Preference-based ranking. The proposal described a scoring system that would rank schedules based on preferences like avoiding morning classes or keeping a day free. We cut this for two reasons. First, we underestimated how much work the core scheduling engine (FR5) and prerequisite validation (FR2) would need, particularly handling multi-section courses and evaluating the

prerequisite trees. Those took up most of the Winter term. Second, the system already returns up to 50 valid schedules that you can flip through visually, which gets you most of the way there even without automated ranking. Building a proper preference UI, a scoring algorithm, and testing it all was more than we could do well in the time we had left.

4 Implementation

This chapter goes through the implementation of each major component. The emphasis is on the algorithms, data structures, and places where we had to make judgment calls.

4.1 Prerequisite Validation Engine

4.1.1 Problem Statement

Given a student's transcript (completed courses with grades) and a target course, determine whether the student satisfies all prerequisites. Prerequisite rules at Carleton come in several forms:

- Simple requirements: "SYSC 3110"
- Conjunctions: "SYSC 3110 and SYSC 3020"
- Disjunctions: "SYSC 3110 or COMP 3005"
- Nested expressions: "(SYSC 3110 and SYSC 3020) or (COMP 3005 and COMP 3007)"
- Minimum grades: "SYSC 2004 with a minimum grade of C+"
- Year standing: "Fourth-year standing"
- Special permission: "Permission of the Department"

4.1.2 Data Structure

Prerequisites are stored as a tree of typed nodes in the `PrerequisiteNode` table. Each node is one of five types:

- **AND**: All children must be satisfied (logical conjunction).
- **OR**: At least one child must be satisfied (logical disjunction).

- **COURSE**: A specific course must be completed, optionally with a minimum grade.
- **YEAR_STANDING**: The student must have reached a specified year level.
- **PERMISSION**: The student must have special permission (treated as a boolean flag).

For example, the prerequisite "SYSC 3110 and (COMP 3005 or COMP 3007)" is represented as:

```
         AND (root)
        /         \
   COURSE          OR
  SYSC3110        /    \
            COURSE    COURSE
           COMP3005  COMP3007
```

4.1.3 Algorithm

The validation algorithm is a recursive depth-first traversal of the tree:

```
evaluateNode(node, transcript):
  case AND:
    return ALL(evaluateNode(child) for child in node.children)
  case OR:
    return ANY(evaluateNode(child) for child in node.children)
  case COURSE:
    hasCourse = node.requiredCourseId in transcript.completedCourses
    if not hasCourse: return false
```

```
    if node.requiredGrade:
      return transcript.grades[node.requiredCourseId] >=
node.requiredGrade
    return true
  case YEAR_STANDING:
    return transcript.currentYear >= node.requiredYear
  case PERMISSION:
    return transcript.hasSpecialPermission
```

The time complexity is O(n) where n is the number of nodes in the prerequisite tree, since each node is visited exactly once. The space complexity is O(h) where h is the height of the tree, due to the recursion stack.

4.1.4 Grade Comparison

Grades are mapped to numbers (A+ = 12, A = 11, down to F = 0) so we can compare them. If a grade cannot be parsed (transfer credits marked "TR" or "SAT," for instance), the system treats the requirement as satisfied. If the system has to guess, we would rather let a student through than incorrectly block them from a course they are probably allowed to take.

4.1.5 Design Decision: Recursive vs. Iterative

We used recursion because prerequisite rules are stored as trees, and recursion is an intuitive way to process that kind of structure. An iterative version was possible, but it would have needed extra code to keep track of which nodes had already been visited, which ones still needed to be checked, and how the child results should be combined at each AND or OR node. That makes the logic

harder to follow. Since prerequisite trees are small (typically under 20 nodes), recursion does not add much risk here. The call depth stays shallow, so there is no realistic chance of running into stack overflow problems.

4.2 Schedule Generation Algorithm

4.2.1 Problem Definition

Given n courses, where each course has multiple sections (lectures, labs, tutorials) at specific times, find all timetable combinations where:

1. Every course has all its required section types covered (e.g., one LEC and one LAB).
2. No two selected sections overlap.

This is a constraint satisfaction problem (CSP).

4.2.2 Algorithm Selection

We considered three approaches:

1. **Brute force**: Generate every possible combination, then filter for valid ones. The search space grows as $O(s^n)$ where s is the average number of section combinations per course and n is the number of courses. Even with modest inputs this becomes infeasible, so we ruled it out.
2. **Genetic algorithm**: Start with a set of candidate schedules, score them, and repeatedly modify and combine them to try to produce better ones over time. We rejected this because our problem size is small enough (4–6 courses in practice) that we can search directly for exact answers. Genetic algorithms are better suited to very large search spaces where finding a good answer is

enough. In our case, we wanted all valid schedules, not just one or two decent ones, and a genetic algorithm could easily miss some of them.

3. **Backtracking with constraint propagation (what we used)**: Explore the possible schedules in an organized way, one course at a time. Whenever a partial schedule already has a time conflict, stop exploring that path immediately and move on to the next one. This lets the algorithm find all valid schedules without wasting time on combinations that are already invalid.

4.2.3 Implementation

The algorithm works in three phases:

Phase 1: Data preparation. For each selected course, fetch all available sections from the database and group them by type (LEC, LAB, TUT). Compute the Cartesian product across section types to get every possible combination for that course. A course with 3 lectures and 2 labs produces 6 LEC+LAB combinations. Before the search even starts, we filter out any combination where a course's own sections conflict with each other (e.g., a lecture and a lab that overlap).

Phase 2: Recursive backtracking. Process courses one at a time. For the current course, try each valid section combination. Check whether any section in the combination conflicts with sections already assigned to previous courses. If there is no conflict, add the combination to the partial schedule and recurse to the next course. If all courses have been assigned, record the schedule as a solution. Then backtrack: remove the last assignment and try the next combination. Stop after finding 50 valid schedules.

Phase 3: Prerequisite check. After generating schedules, optionally validate that the student meets the prerequisites for each course using the validator from Section 4.1. This information goes into the API response so the frontend can show prerequisite warnings.

4.2.4 Conflict Detection

Two sections conflict if they are on the same day and their time intervals overlap. With start and end times in minutes since midnight, the overlap condition is:

$$\text{start}_1 < \text{end}_2 \wedge \text{start}_2 < \text{end}_1$$

This is O(1) per pair of time slots. Day strings like "MWF" are expanded into individual days before comparison.

4.2.5 Performance

Courses	**Avg. Sections/ Course**	**Estimated Search Space**	**Observed Time**
1	8	8	$<$ 100ms
2	6	48	$<$ 200ms
3	5	240	$<$ 500ms
4	4	960	$<$ 1s
5	3	2,430	$<$ 2s

Table 4: Schedule Generation Performance

Pruning eliminates roughly 99.99% of the search space for typical inputs. Most partial schedules get abandoned early because of conflicts, so the algorithm rarely comes close to exploring the full space. The worst-case complexity is $O(b^d)$ where b is the average number of section combinations per course and d is the number of courses, but in practice the pruning makes this much faster.

We cap requests at 8 courses to avoid unusually large inputs that would create a very large search space and make schedule generation take too long.

4.3 Academic Audit Parsing

4.3.1 HTML Parsing

The main audit parsing method processes the HTML file that students can save from the Carleton Central academic audit page. The parser uses regular expressions to extract:

- **Student ID**: Matched by `(\d{9})`.
- **Student name**: Matched from header text in "LastName, FirstName" format.
- **Program**: Extracted from "Bachelor of Engineering - …" text.
- **Completed courses**: Pulled from HTML table rows with class `takenCourse`, capturing the course code, term, and grade.
- **Current courses**: Same structure, identified by grade value "CUR".
- **Degree requirements**: Extracted from `subrequirement` div elements, with credit counts, completion status, and the courses that satisfy each requirement.

We used regular expressions instead of a DOM parsing library (Cheerio was in the proposal) after looking at the actual HTML. Carleton's audit pages are generated by a consistent server-side system and follow predictable patterns. A DOM parser would be more robust against structural changes, but it would add a dependency for something that regex handles fine given how regular the input is.

4.3.2 PDF Parsing

PDF parsing was a secondary objective in the proposal because PDFs are inherently harder to extract structured data from. We first tried pdf-parse (a JavaScript library), but the text extraction was unreliable for the multi-column layout in Carleton audit PDFs. Text would come out jumbled or with columns interleaved.

What worked was `pdftotext` (from the Poppler library) with the `-layout` flag, which preserves the spatial arrangement of text on the page. The backend writes the uploaded PDF to a temp file, runs `pdftotext` as a child process, reads the output, and parses it with regex. Temp files are cleaned up immediately after, even if an error occurs.

The PDF parser matches course lines like `"21FA  CHEM 1101  0.50  A+"` using:

```
/^\s*(\d{2}(?:FA|WN|SU))\s+([A-Z]{3,4})\s+(\d{4})\s+([\d.]+)\s+([A-
Z+\-]+)/gm
```

This pattern looks for course lines that follow the usual audit format. It pulls out the term, subject code, course number, credit value, and grade. That gives the parser the exact pieces it needs from each course entry without having to process the whole line manually.

It handles repeated courses (student retook a course) by keeping the most recent grade, and drops courses with F, UNS, or NR grades.

4.3.3 Design Decision: `pdftotext` vs. JavaScript Library

Using a system-level CLI tool instead of a JavaScript library was not what we planned. But Poppler's text extraction is significantly better than the JS

alternatives we tried, and it handles multi-column layouts correctly. The downside is a system dependency: `pdftotext` has to be installed on the host machine, which hurts portability. For a local development tool, that was an acceptable tradeoff. For production, we would either bundle it in a Docker container or revisit the JS libraries.

4.4 Backend API

The Express backend has four groups of endpoints:

Method	Endpoint	Description
GET	`/api/courses/search?q=&limit=`	Search courses by code or title
GET	`/api/courses/:courseId`	Get course details with prerequisites
GET	`/api/courses/:courseId/sections`	Get all sections for a course
GET	`/api/students/:studentId`	Get student profile and completions
POST	`/api/students/:studentId/validate`	Validate prerequisites for a course
GET	`/api/students/:studentId/eligible`	Get courses student is eligible for
POST	`/api/schedules/generate`	Generate conflict-free schedules
POST	`/api/audit/upload`	Upload and save audit data
POST	`/api/audit/parse`	Parse audit without saving (preview)
POST	`/api/audit/suggest-courses`	Suggest courses from remaining requirements

Table 5: REST API Endpoints

The server runs on port 3001 with CORS configured for the frontend dev server. File uploads use the Multer middleware with a 10 MB limit and file type validation (HTML and PDF only). All route handlers catch errors and return appropriate HTTP codes (400 for bad input, 404 for missing resources, 500 for server errors) instead of letting unhandled exceptions crash the process.

Appendix B includes example API requests and responses for the schedule generation endpoint.

4.5 Frontend Interface

The frontend is a three-step wizard:

4.5.1 Page 1: Academic History Import

The first page lets users upload an academic audit (HTML or PDF) or skip ahead to manual course selection. When a file is uploaded, it goes to the `/api/audit/parse` endpoint, and the page shows the extracted info: student name, ID, program, completed courses, current courses, and remaining requirements. The user confirms the parsed data before moving on. If parsing fails or the user does not have an audit file, they can proceed without it, and prerequisite indicators are hidden.

4.5.2 Page 2: Course Selection

This page has a search bar (backed by `/api/courses/search`) with 300ms debounce to avoid hammering the API on every keystroke. Users can also add courses from their remaining requirements list, which comes from the audit parsing step. Each selected course shows its prerequisite status: green if met, amber if not, hidden if there is no audit data. Elective categories (like "Computer Science Elective" or "Complementary Studies") show up as selectable tokens. Selecting one tells the system to auto-pick an appropriate course during schedule generation.

4.5.3 Page 3: Schedule Review

Generated schedules appear in a weekly timetable grid. Courses are color-coded, and each block shows the course code, section code, time, and location. You flip between schedules with prev/next buttons ("Schedule 1 of 50"). A sidebar lists all sections in the current schedule with their CRNs, and each CRN can be copied to the clipboard for entry into Carleton Central.

If elective categories were selected, the sidebar also has an elective panel where you can browse available courses for each category, see which ones have sections and met prerequisites, and swap electives. Swapping regenerates the schedules.

4.5.4 UI Technology

The frontend uses 48 shadcn/ui components (built on Radix UI) for the UI elements, TailwindCSS v4 for styling, and Lucide icons. Application state (current step, parsed audit data, selected courses, generated schedules, elective selections) lives in the top-level `App.tsx` component and gets passed down as props, following React's one-way data flow.

5 Testing and Evaluation

5.1 Testing Approach

We did not use an automated test framework. No Jest, no Vitest. This is probably the biggest shortcoming of the project, and Chapter 6 discusses it. All testing was done manually during development.

5.1.1 Manual Functional Testing

We tested each feature by hand against known inputs:

Prerequisite Validation: We ran the validator against a range of scenarios:

- Courses with a single prerequisite (e.g., SYSC 3110 requires SYSC 2004).
- AND prerequisites (e.g., needing both SYSC 3110 and SYSC 3020).
- OR prerequisites (e.g., COMP 3005 or COMP 3007).
- Nested expressions (e.g., "(A and B) or (C and D)").
- Minimum grade requirements (e.g., SYSC 2004 with minimum C+).
- Year-standing requirements (e.g., fourth-year standing).
- Courses with no prerequisites (should always pass).

The CLI tool was invaluable for this. We could test dozens of scenarios in a few minutes without touching the browser.

Schedule Generation: We tested with:

- Single-course inputs (trivial case: should return all sections).
- Two-course inputs with known section times (manually verified the output).
- Multi-section courses (LEC+LAB) to check that all valid combinations came through and internal conflicts were filtered.
- Stress tests with 6–8 courses for performance and the 50-schedule cap.

- Edge cases: courses with no available sections, sections at identical times.

Audit Parsing: We tested the HTML parser with real academic audits from team members. The PDF parser was tested with the same audits saved as PDFs. We manually compared the outputs to verify that courses, grades, and student info were extracted correctly.

Frontend Integration: We tested the full flow end-to-end in the browser: upload audit, select courses, generate schedules, browse timetables, copy CRNs.

5.1.2 CLI Testing Tool

The project includes a command-line interface (`cli/` directory) that lets you test backend logic without the web app. It has commands for:

- Validating prerequisites for any course against a given transcript.
- Parsing an audit file and printing the extracted data.
- Running the schedule generator for a set of courses and printing results.

This was one of the better decisions we made. Having immediate feedback on algorithm behavior without the overhead of the full web stack saved a lot of time.

5.2 Requirements Verification

Requirement	Verification Method	Result
FR1: Audit Parsing	Tested with team members' actual audits (HTML and PDF)	Pass — correct extraction of courses, grades, and student info
FR2: Prerequisite Validation	CLI testing with known prerequisite trees	Pass — correctly evaluates AND, OR, COURSE, YEAR_STANDING nodes
FR3: Manual Course Entry	UI testing: search, select, remove courses	Pass — courses can be searched, added, and removed
FR4: Elective Recommendation	Tested suggest-courses endpoint with audit data	Partial — suggestions returned but not fully integrated with UX
FR5: Schedule Generation	Tested with 1–8 course combinations	Pass — all valid schedules generated, no false conflicts
FR6: Preference Ranking	N/A	Not implemented
FR7: Timetable View	Visual inspection of generated schedules in browser	Pass — correct time placement and color coding
FR8: Error Feedback	Tested with invalid inputs, missing sections, unmet prereqs	Pass — appropriate error messages and warnings displayed

Table 6: Functional Requirements Verification Matrix

5.3 Limitations of Testing Approach

The lack of automated tests is the biggest testing gap. Without a test suite, there is no regression protection. A change to one component can break another and

nobody notices until someone happens to test that path manually. We relied on code review and manual testing to catch errors, which works but does not scale. If the project were to continue, we would add:

1. Unit tests for the prerequisite evaluator (it is a pure function with clear inputs and outputs, easy to test).
2. Unit tests for conflict detection.
3. Integration tests for the API endpoints using a test database.
4. End-to-end tests with something like Playwright or Cypress.

We also never did the User Acceptance Testing with student volunteers that the proposal described. The system was tested only by the four of us. We know it too well to evaluate usability objectively. Even informal testing with a few peers outside the team would have been useful.

6 Reflections

This chapter is a straightforward look at the project: what worked, what did not, and what we would change if we had to do it again.

6.1 Achievement of Original Objectives

The proposal said we would build an application that automates requirement analysis, validates prerequisites, generates conflict-free timetables, and ranks them by preference. In the end, we got three of those four done:

1. **Automated requirement analysis**: The system parses academic audits in HTML and PDF format, extracts completed courses and degree requirements, and figures out what courses a student still needs. Done.
2. **Prerequisite validation**: The tree-based validator handles AND/OR logic, grade thresholds, year-standing requirements, and the various other patterns in the Carleton calendar. Done.
3. **Conflict-free timetable generation**: The backtracking solver finds all valid schedules for a course selection, handles multi-section courses, and finishes in under 2 seconds for typical inputs (well within the 10-second target from NFR1). Done.
4. **Preference-based ranking**: Not implemented. Users browse unranked schedules manually.

The elective recommendation feature (FR4) was only partly done. The backend can suggest courses from remaining requirements, and the frontend shows elective categories. But that workflow was added late and it still feels rougher than the rest of the application.

6.2 What Changed and Why

Quite a bit changed between the proposal and the final system:

Technology stack: The biggest change was going from Next.js to React+Vite+Express. Section 3.2.1 covers the details. The short version is that we needed the split so four people could work without constantly stepping on each other, and that part worked.

Audit parsing approach: The proposal said Cheerio for HTML and a JavaScript library for PDF. We ended up using regex for HTML and `pdftotext` for PDF. Neither was the plan, but both were driven by what actually worked when we tried parsing real audit files. The JS PDF libraries produced garbled output. The HTML was regular enough that regex worked fine, so that is what we used.

UI library: The proposal mentioned Material-UI or Ant Design. The final version used Radix UI/shadcn instead. That was mostly a preference call and it did not change the core functionality.

Deployment: The proposal said we would deploy to Vercel. We never did. Partly because the frontend/backend split made Vercel less convenient (it is optimized for Next.js), and partly because deployment was not a priority given everything else that still needed doing. This means NFR6 (HTTPS) was not met.

Descoped features: FR6 (ranking) got cut. We underestimated how much time the core features would take, especially multi-section schedule generation and the prerequisite tree system.

6.3 Team Dynamics

For the most part, the team worked well together. Splitting responsibilities by strength (Hakeem on data acquisition, Pietro on database schema and prerequisite logic, Anas on the scheduling algorithm, Safi on PDF parsing and search optimization) kept us out of each other's way and let people focus on the parts they were strongest at. Weekly meetings were enough to keep everyone lined up.

That said, things got shakier during the Winter term, especially around integrating the frontend with the backend API. Without a proper task tracker (we never set up GitHub Issues or anything like it), it was sometimes unclear who owned what. A few tasks fell through the cracks because everyone assumed someone else was handling them. In the last few weeks, when multiple features were being finished simultaneously, better tracking would have helped.

6.4 Hindsight Decisions

If we started over knowing what we know now:

Write tests from the beginning. The lack of automated tests made refactoring scary and led to bugs that only surfaced during manual testing late in the process. Even basic unit tests for the prerequisite evaluator and conflict detection would have saved us time in the long run. We kept saying we would add tests later, and later never came.

Use a task tracker. GitHub Issues with a project board would have been easy to set up and would have made ownership clearer. Our shared-list approach worked until it didn't.

Cut features earlier. By the end of the Fall term, we already had a pretty good sense that ranking was slipping. We still waited too long to say that out loud and cut it. Making that decision earlier would have given us more time to polish the parts we actually finished.

Do user testing. The proposal described UAT with student volunteers. We never did it. Even asking a few classmates to try the app and watching where they got confused would have been useful.

Set up Docker. We occasionally ran into environment issues: different Node.js versions, missing system dependencies like `pdftotext`. A Docker Compose file would have prevented all of that.

6.5 Overall Assessment

The Smart Scheduler works. You can upload an audit, it figures out what you have taken, you pick courses, and it generates valid schedules. The prerequisite tree and the backtracking solver are the parts we feel best about. Those are the pieces that solve the real planning problem.

But the project has gaps. No automated tests. Two features that never made it past the proposal. No formal usability evaluation. These are not from lack of effort. We underestimated how much work the core features would take and did not leave enough margin for everything else.

What we got out of this project was not just the prototype. We also learned a lot from the process around it: making design decisions with tradeoffs, changing plans when our assumptions were wrong, coordinating work across a team, and

being honest when things did not go the way we expected. We wanted the report to capture that part too.

7 References

1. I. Sbeity, M. Dbouk, and H. Kobeissi, "Combining the analytical hierarchy process and the genetic algorithm to solve the timetable problem," *arXiv preprint*, Sep. 2014. Available: https://arxiv.org/abs/1409.2650
2. L. Zhang, *Solving the timetabling problem using constraint satisfaction programming*, MInfoSys thesis, University of Wollongong, Australia, 2005. Available: https://ro.uow.edu.au/articles/thesis/27827784
3. T. Schaub, "Answer set programming as a modeling language for course timetabling," *Theory and Practice of Logic Programming*, vol. 8, no. 5, pp. 541–564, Sep. 2008.
4. R. K. Thakur, N. K. Agrawal, and P. Kumar, "A practical approach to college timetable scheduling," *Mathematical Modeling and Computing*, vol. 11, no. 3, pp. 710–719, 2024.
5. T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA: MIT Press, 2022.
6. R. C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Upper Saddle River, NJ: Prentice Hall, 2018.
7. M. Fowler, *Patterns of Enterprise Application Architecture*. Boston, MA: Addison-Wesley, 2003.
8. PostgreSQL Global Development Group, *PostgreSQL 16 Documentation*, 2024. Available: https://www.postgresql.org/docs/16/
9. Express.js Contributors, *Express.js Documentation*, 2024. Available: https://expressjs.com/

10. Prisma Data, Inc., *Prisma Documentation*, 2024. Available: https://www.prisma.io/docs/

H Appendix A: Original Project Proposal

The original project proposal (G67_SmartScheduler.pdf, October 17, 2025) is included in full after this appendix page. It defines the initial functional and non-functional requirements, the proposed technology stack, the system architecture, the project timeline, and the risk assessment.

Key sections of the proposal that are referenced throughout this report include:

- **Section 3.1**: Functional Requirements (FR1–FR9)
- **Section 3.2**: Non-Functional Requirements (NFR1–NFR6)
- **Section 3.3**: Success Metrics
- **Section 4.2**: Original Technology Stack (Next.js 14, Cheerio, Material-UI, Vercel)
- **Section 8.1**: Phased Timeline
- **Section 9.0**: Risks and Mitigation Strategies

The following pages reproduce the original proposal as submitted.

I Appendix B: API Reference

The complete REST API is documented in Section 4.4. The base URL is `http://localhost:3001/api`. All endpoints return JSON responses. Error responses follow the format:

```
{
  "error": "Error Type",
  "message": "Human-readable description"
}
```

I.0.1 Schedule Generation Request Example

```
POST /api/schedules/generate
{
  "courseIds": ["SYSC4907", "SYSC4001", "ELEC4705"],
  "maxSchedules": 50,
  "completedCourses": ["SYSC3110", "SYSC3020", "COMP3005"]
}
```

I.0.2 Schedule Generation Response Example

```
{
  "schedules": [
    {
      "id": 1,
      "selections": [
        {
          "courseId": "SYSC4907",
```

```
          "section": {
            "crn": "30123",
            "sectionCode": "A",
            "sectionType": "LEC",
            "days": "TR",
            "startTime": "1135",
            "endTime": "1325",
            "instructor": "Talim, J."
          }
        }
      ],
      "hasConflicts": false,
      "prerequisiteInfo": {
        "SYSC4907": {
          "isValid": true,
          "expressionEvaluated": "..."
        }
      }
    }
  ],
  "count": 42,
  "requested": 3,
  "maxSchedules": 50
}
```

J Appendix C: Database Statistics

Metric	Value
Total courses in database	348
Courses with prerequisite trees	322
Courses with section data	1,836
Total sections	7,680
Prerequisite node count	2,000
Maximum prerequisite tree depth	4 levels
Average sections per course	4.2